Strings & Pointers

Arrays

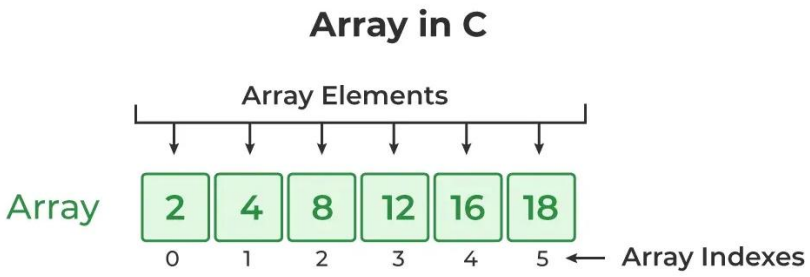
An array is a collection of elements of the same data type stored under a single variable name.

For example, instead of declaring five variables to store five marks, we can use a single array.

int marks[5] = {80, 70, 90, 60, 85};

One-Dimensional Array (1D Array)

A one-dimensional array stores elements in a single row.



Declare and print an array

```
#include <stdio.h>
int main() {
    int a[5] = {10, 20, 30, 40, 50};
    int i;
    for (i = 0; i < 5; i++) {
        printf("%d ", a[i]);
    }
    return 0;
}
```

10 20 30 40 50

Array Operations

Common operations performed on arrays include:

- Traversal – visiting every element.
- Insertion – adding an element.
- Deletion – removing an element.
- Searching – finding an element.
- Updating – changing an element.
- Sorting – arranging elements in order.

Traversal of an array

Traversal means visiting each element of an array one by one.

```
#include <stdio.h>
int main() {
    int a[4] = {5, 10, 15, 20};
    int i;
    for (i = 0; i < 4; i++) {
        printf("%d\n", a[i]);
    }
    return 0;
}
5
10
15
20
```

Update Array Element

```
#include <stdio.h>
int main() {
    int arr[5] = {2, 4, 8, 12, 16};
    arr[0] = 1;
    printf("%d", arr[0]);
    return 0;
}
1
```

Size of an Array

```
#include <stdio.h>
int main() {
    int arr[5] = {2, 4, 8, 12, 16};
    int size = sizeof(arr)/sizeof(arr[0]);
    printf("%d", size);
    return 0;
}
5
```

Find the sum of array elements

```
#include <stdio.h>
int main() {
    int a[4] = {10, 20, 30, 40};
    int i, sum = 0;

    for (i = 0; i < 4; i++) {
        sum = sum + a[i];
    }
```

```
    printf("Sum = %d", sum);
    return 0;
}
Sum = 100
```

Search for an element

```
#include <stdio.h>
int main() {
    int a[5] = {10, 20, 30, 40, 50};
    int i, key = 30;
    for (i = 0; i < 5; i++) {
        if (a[i] == key) {
            printf("Element found at index %d", i);
            return 0;
        }
    }
    printf("Element not found");
    return 0;
}
Element found at index 2
```

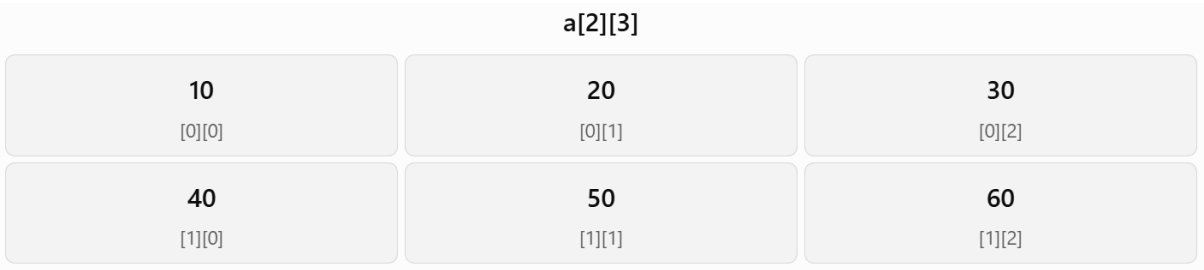
Multi-Dimensional Arrays

A multidimensional array stores data in two or more dimensions.

2D Array

A 2D array can be understood as a collection of 1D array

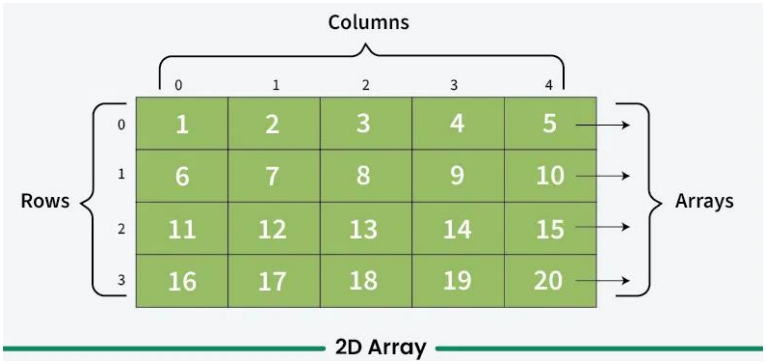
```
int a[2][3] = {
    {10, 20, 30},
    {40, 50, 60}
};
```

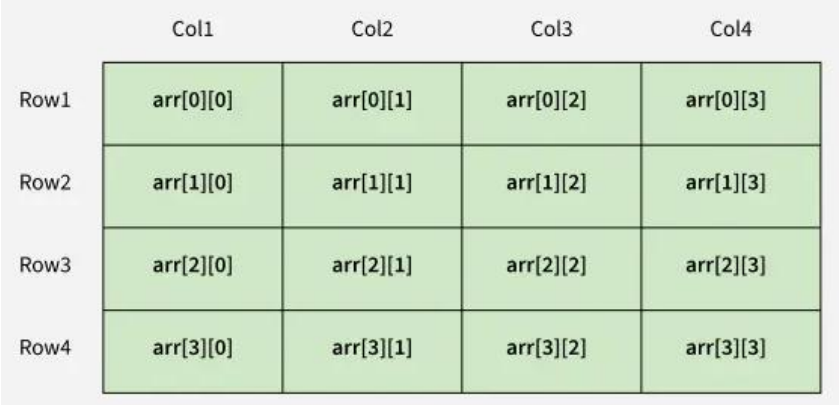


For example:

- a[0][0] = 10

- a[0][1] = 20
- a[1][0] = 40
- a[1][2] = 60





Print a 2D array using nested loops

```
#include <stdio.h>
int main() {
    int a[2][3] = {
        {10, 20, 30},
        {40, 50, 60}
    };
    int i, j;
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 3; j++) {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }
    return 0;
}
10 20 30
40 50 60
```

Read and print a 2D array

```
#include <stdio.h>
```

```
int main() {
    int a[2][2];
    int i, j;
    printf("Enter 4 numbers:\n");
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++) {
            scanf("%d", &a[i][j]);
        }
    }
    printf("Array elements:\n");

    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++) {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }
    return 0;
}
Enter 4 numbers
1 2 3 4
Output
Array elements:
1 2
3 4
```

Initialization of 2D Array

```
int arr[3][4] = {
    {0, 1, 2, 3},
    {4, 5, 6, 7},
    {8, 9, 10, 11}
};
            or
int arr[3][4] = {0, 1 ,2 ,3 ,4 , 5 , 6 , 7 , 8 , 9 , 10 , 11};
```

Three-Dimensional Array (3D Array)

A 3D array can be understood as a collection of 2D arrays.

```
int a[2][2][2] = {

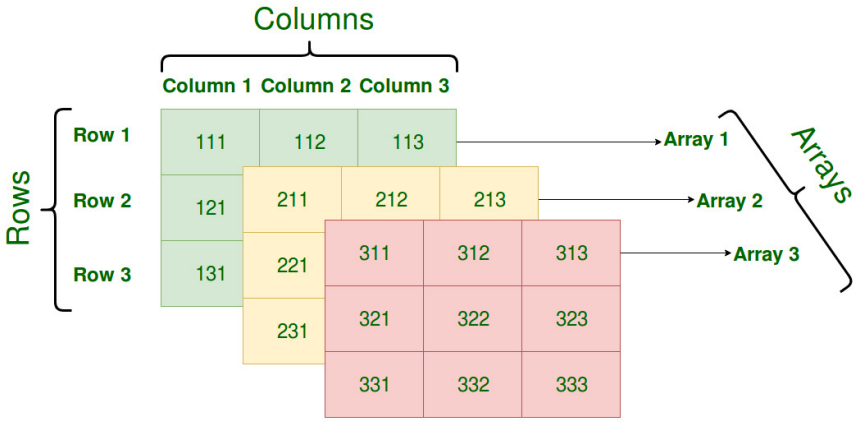
    {{1, 2}, {3, 4}},

    {{5, 6}, {7, 8}}

};
```

To access an element, use three indices:

- a[0][0][0] = 1
- a[0][1][0] = 3
- a[1][0][0] = 5
- a[1][1][1] = 8




```
#include <stdio.h>
int main() {
    int a[2][2][2] = {
        {{1, 2}, {3, 4}},
        {{5, 6}, {7, 8}}
    };
    int i, j, k;
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++) {
            for (k = 0; k < 2; k++) {
                printf("%d ", a[i][j][k]);
            }
            printf("\n");
        }
        printf("\n");
    }

    return 0;
}
1 2
3 4

5 6
7 8
```

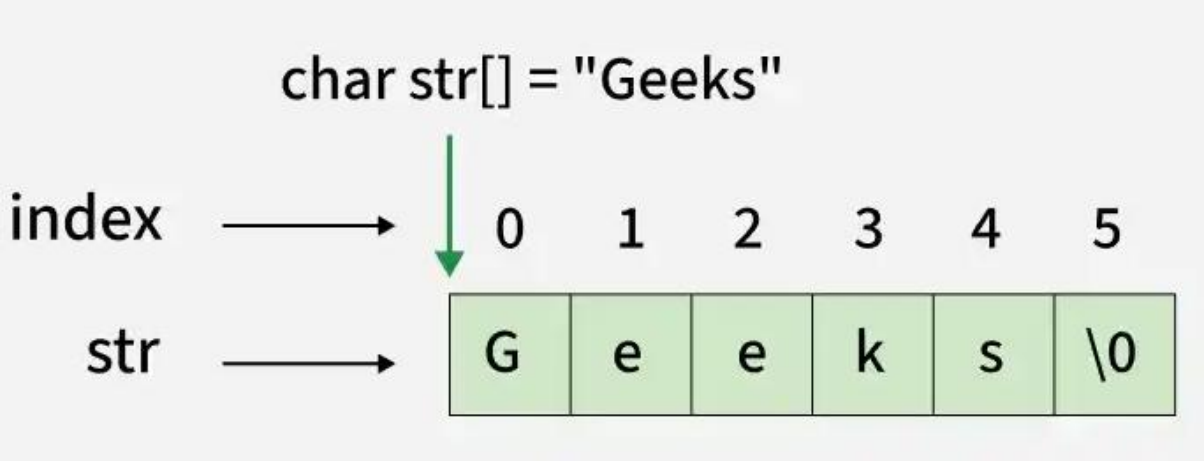
Initialization of 3D Array

```
int arr[2][3][2] = {0, 1, 2, 3, 4, 5, 6, 7 , 8, 9, 10, 11};
or
int arr[2][3][2] = {
1, 2, 3, 4, 5, 6,
7, 8, 9, 10, 11, 12
```

```
};
or
int arr[2][3][2] = { { { 1, 1 }, { 2, 3 }, { 4, 5 } },
{ { 6, 7 }, { 8, 9 }, { 10, 11 } } };
or
int arr[2][3][2] = {
{
{1, 1},
{2, 3},
{4, 5}
},
{
{6, 7},
{8, 9},
{10, 11}
}
};
```

Strings

A string in C is a sequence of characters stored in a character array and terminated by the null character '\0'




```
#include <stdio.h>
int main() {
    char str[] = "Computer";
    printf("The string is: %s\n", str);
    return 0;
}
```

The string is: Computer

Accessing Characters

```
#include <stdio.h>
int main() {
     char str[] = "Computer";
```

```
    printf("%c", str[0]);
    return 0;
}
```
C

Update

```
#include <stdio.h>
int main() {
    char str[] = "Mohan";
    str[0] = 'R';
    printf("%s", str);
    return 0;
}
```
Rohan

String Length

```
#include <stdio.h>
#include<string.h>
int main() {
    char str[] = "Software";

    printf("%d", strlen(str));
    return 0;
}
```
8

String Input

```
#include<stdio.h>
int main() {
    char str[6];
    scanf("%s",str);
    printf("%s",str);
    return 0;
}
```
Undefined behaviour

```
#include<stdio.h>
int main() {
    char str[6];
    scanf("%5s",str);
    printf("%s",str);
    return 0;
}
```
Computer
Output
compu

RR BB
Output
RR

Using fgets()

```
#include <stdio.h>
int main() {
    char str[20];
    fgets(str, 20, stdin);
    printf("%s", str);
    return 0;
}
```

Ashoka Rajan
Ashoka Rajan

String Library Functions

Function	Purpose	Example
`strlen()`	Finds string length	`strlen("Hello")` → `5`
`strcpy()`	Copies one string to another	`strcpy(b, a)`
`strcat()`	Joins two strings	`strcat(a, b)`
`strcmp()`	Compares two strings	`strcmp(a, b)`

```
#include <stdio.h>
#include <string.h>
int main() {
    char a[30] = "Hello";
    char b[30] = "World";
    printf("Length = %d\n", (int)strlen(a));
    strcpy(b, a);
    printf("After copying: %s\n", b);
    strcat(a, " Hai");
    printf("After joining: %s\n", a);
    printf("Comparison = %d\n", strcmp("c", "b"));
    return 0;
}
```

Length = 5
After copying: Hello
After joining: Hello Hai
Comparison = 1